

EpiDiagnostix-Mayab — Guía de modelos y proceso de desarrollo

Sistema de extracción clínica 100% offline para dispositivos Android de gama media-baja. Convierte texto libre (dictado de voz o escritura) en campos estructurados de una consulta médica.

Campos que el sistema extrae

Campo	Tipo	Ejemplo
edad	int	45
sexo	"M" / "F"	"M"
peso_kg	double	72.5
talla_cm	double	168.0
presion_sistolica	int	120
presion_diastolica	int	80
glucosa_mg_dl	int	95
temperatura_c	double	36.7
frecuencia_cardiaca_bpm	int	78
duracion_sintomas_dias	int	3
categoria_sintoma	String	"Gastrointestinal"

Cronología de modelos entrenados

Modelo 0 — Demostración SpaCy mínima

Script: `entrenar.py`

Salida: ningún archivo guardado (solo consola)

El primer experimento del proyecto. SpaCy en blanco sobre español, solo 5 frases de entrenamiento, 2 etiquetas (`EDAD`, `SINTOMA`), 20 epochs. Su único propósito fue confirmar que el entorno de Python/spaCy funcionaba y que la idea de extraer entidades de texto clínico era viable antes de invertir tiempo en un dataset real.

No se usa en producción; fue el "hola mundo" conceptual del proyecto.

Modelo 1 — SpaCy NER clínico (producción local)

Script: `entrenar_ner.py`

Salida: `models/ner_clinico_es/`

Framework: spaCy 3.8, modelo en blanco `es`

Etiquetas: 11 entidades (`EDAD`, `SEXO`, `PESO_KG`, `TALLA_CM`, `PRESION_SIS`, `PRESION_DIA`, `GLUCOSA`, `TEMPERATURA`, `FREC_CARD`, `DURACION`, `CATEGORIA`)

El primer modelo real. Se construyó un generador sintético de frases clínicas (`hacer_texto`) que combina plantillas de estilo con valores aleatorios del CSV `consultas_clinicas.csv`. Se entrenó con 60 epochs, dropout 0.35, early stopping de 10 epochs de paciencia, y split 80/10/10.

Por qué se hizo: spaCy era la ruta más rápida para tener un modelo NER funcional. El problema fue que spaCy genera modelos de varios MB con dependencias pesadas — no exportable a TFLite y no ejecutable en Flutter sin un servidor backend.

Por qué se descartó para producción móvil: spaCy no tiene soporte TFLite. Requiere un microservicio Python siempre encendido, lo que viola el requisito de 100% offline en el dispositivo.

Modelo 2 — Isolation Forest (detección de anomalías / brotes)

Script: `analisis_medico.py`

Salida: `models/isolation_forest.joblib`, `models/scaler_if.joblib`, `models/isolation_forest_meta.joblib`

Framework: scikit-learn, `IsolationForest`

Features: 10 columnas numéricas (`edad`, `peso_kg`, `talla_cm`, `presion_sistolica`, `presion_diastolica`, `glucosa_mg_dl`, `temperatura_c`, `frecuencia_cardiaca_bpm`, `duracion_sintomas_dias`, `categoria_sintoma_enc`)

Componente de vigilancia epidemiológica, paralelo al extractor NER. Recibe los campos ya extraídos y detecta si el conjunto de signos vitales es anómalo (posible brote, valores fuera de rango clínico esperado). Se entrenó con `contamination` óptimo encontrado por barrido (≈ 0.03), 200 árboles, `random_state=42`.

Métricas (sobre set de validación): precision ≈ 0.67 , recall ≈ 0.75 , F1 ≈ 0.71 .

Papel en el sistema: después de que el NER extrae los campos de una consulta, el Isolation Forest evalúa si ese perfil de signos vitales es estadísticamente inusual. Es el módulo de alerta de brotes, no el de extracción de campos.

Modelo 3 — BiLSTM NER v1 (primer modelo TFLite)

Script: `generar_dataset_tflite.py` → `entrenar_tflite_ner.py`

Salida: `models/ner_tflite.keras`

Framework: TensorFlow 2.15 / Keras

Arquitectura: `Embedding(1981, 64) → BiLSTM(64) → BiLSTM(32) → TimeDistributed(Dense(14, softmax))`

Vocabulario: 1 981 tokens

Etiquetas BIO: 14 (`O` + `B` - `I` - para los 11 campos)

MAX_LEN: 40 tokens

Parámetros totales: 234 958

Epochs: 30

F1 macro en test sintético: 1.00

Por qué esta arquitectura:

- BiLSTM ve el contexto en ambas direcciones — necesario para resolver "108/65" donde el modelo debe saber que 108 es sistólica y 65 es diastólica dependiendo de lo que apareció antes.
- Embedding ligero (64 dims) mantiene el modelo pequeño para TFLite.
- `TimeDistributed(Dense)` produce una probabilidad por token — es un modelo de etiquetado de secuencia (NER), no de clasificación de frase completa.
- MAX_LEN=40 fue determinado analizando la distribución de longitudes del dataset: el percentil 99 estaba bajo 40 tokens.

Por qué se entrenó un v2: el F1=1.0 en test sintético era engañoso — el dataset v1 tenía poca diversidad de estructura de frase. Al evaluar sobre frases reales (SET2/SET3) el rendimiento era insuficiente. Se necesitaba más variedad de plantillas y un set ciego de evaluación para medir generalización real.

Modelo 4 — BiLSTM NER v2 (modelo final validado)

Script: `generar_dataset_tflite_v2.py` → `entrenar_tflite_ner_v2.py`

Salida: `models/ner_tflite_v2.keras`

Framework: TensorFlow 2.15 / Keras

Arquitectura: idéntica al v1 (mismos hiperparámetros)

Vocabulario: 2 196 tokens (expandido)

Epochs: 32

F1 macro en test sintético: 0.9118

SET2 (50 casos reales): 90.0%

SET3 (20 frases ciegas, nunca vistas): 87.8%

Qué cambió respecto al v1:

Aspecto	v1	v2
Estilos de frase	10 plantillas originales	10 originales + 20 estructuras nuevas
Vocabulario de sexo	reducido	expandido: masculino, caballero, dama, varon...
Separadores de presión	solo /	también sobre, x, espacios variables
Set de evaluación	solo test sintético	+ 55 casos de prueba + 20 frases ciegas escritas a mano
Class weights	no	sí — tokens 0 con peso 0.3× para enfocarse en entidades

Por qué el F1 bajó de 1.0 a 0.91 en test sintético: el dataset v2 tiene más variación real, así que el modelo ya no memoriza perfectamente las plantillas. Esto es correcto — el 0.91 sintético con 90% en casos reales es más honesto que el 1.0 que no generalizaba.

Este es el modelo de producción.

Exportación TFLite — modelo on-device

Script: `exportar_tflite.py`

Salida: `models/ner_tflite_v2.tflite`

Tamaño: 757 KB

Cuantización: dynamic range quantization (solo pesos, sin datos de calibración)

Ops: built-in ops estándar — sin SELECT_TF_OPS / sin flex delegate

El problema que hubo que resolver:

`ner_tflite_v2.keras` usa `mask_zero=True` en el Embedding. En TF 2.15 eso genera un nodo `cond/while` (`while_loop`) que el conversor TFLite no puede representar con ops nativas. El error era:

```
Node 'cond/while' has 14 outputs but _output_shapes specifies 44
```

Solución: se reconstruyó el grafo de inferencia con `unroll=True` y `mask_zero=False` — parámetros que no modifican los pesos entrenados, solo el grafo computacional. Se copiaron los pesos capa por capa desde el modelo original y se verificó que las predicciones eran idénticas antes de convertir. Resultado: TFLite con ops nativas, compatible con `tflite_flutter ^0.10.4` sin configuración especial de Android.

Paridad Keras → TFLite: 24/25 casos idénticos en etiquetas, 25/25 en campos extraídos. El único caso diferente (C08) tiene 2 tokens con etiqueta distinta por cuantización, ambos sin impacto en la extracción (un token de texto etiquetado como temperatura no se puede parsear como número; una etiqueta `I-` que el extractor ignora por diseño).

Reglas de post-procesamiento BIO

El modelo BiLSTM comete errores sistemáticos en tres patrones concretos. En lugar de reentrenar para corregirlos, se aplican tres reglas deterministas **después** de la inferencia:

R2 — Dígito atrapado en span SEXO → B-EDAD

Patrón: el modelo etiqueta `"18 anos"` como `B-SEXO I-SEXO` porque aprendió que SEXO y EDAD aparecen juntos. Si hay un token de género real antes (`masculino`, `mujer`, etc.) y el siguiente token al número es `"anos"`, el número es la edad, no parte de SEXO.

```
Entrada:  masculino/B-SEXO  18/B-SEXO  anos/I-SEXO
Salida:   masculino/B-SEXO  18/B-EDAD  anos/O
```

R1 — Segunda B-EDAD con contexto de duración → B-DURACION

Patrón: cuando hay `"Masculino de 18 años... cuadro de 3 días"`, el modelo etiqueta ambos números como B-EDAD. El segundo número cuya ventana de ± 4 tokens contiene palabras de duración (`días`,

evolucion, cuadro, desde, hace, lleva) es realmente la duración.

```
Entrada: 18/B-EDAD ... 3/B-EDAD días
Salida: 18/B-EDAD ... 3/B-DURACION días
```

R3 — Token SEXO sin entrada válida en SEXO_MAP → ignorar

Patrón: al extraer el campo **sexo**, se busca el primer token del span con etiqueta SEXO o I-SEXO que esté en el vocabulario de género conocido (**masculino**→M, **hombre**→M, **mujer**→F, **femenina**→F, etc.). Si ninguno está en el mapa, el campo **sexo** no se escribe.

Impacto medido antes de portar a Dart:

Configuración	SET2 (50 casos)	SET3 (189 campos)
Solo modelo, sin reglas	86%	93.1%
R1 + R2	+2 casos	sin regresión
R1 + R2 + R3	sin regresión	94.7%

Pipeline completo on-device (Flutter/Dart)

```

texto (voz o escritura)
  |
  ▼
tokenize(text)                                ←
flutter_bridge/lib/services/clinical_tokenizer.dart
  regex: \d+[.,]\d+ | \d+ | [a-z]+
  .lower() → ASCII-only (normalización NFD)
  |
  ▼
tokensToIds(tokens, vocab)                    ← padding a MAX_LEN=40 con <PAD>=0
  |
  ▼
TFLite BiLSTM NER                            ← models/ner_tflite_v2.tflite
  Input: [1, 40] int32
  Output: [1, 40, 14] float32
  |
  ▼
argmax por token → labels_raw[]
  |
  ▼
applyRules(tokens, labels_raw)                ←
flutter_bridge/lib/services/bio_post_processor.dart
  aplica R2 primero, luego R1
  |
  ▼
extractFields(tokens, labels_pp)              ← bio_post_processor.dart
  R3 integrado en el extractor

```

↓
 Map<String, dynamic> ← campos listos para la UI / base de datos

Clase principal: `flutter_bridge/lib/services/tflite_extractor.dart` —
`NerExtractor.infer(String text)`

Golden sets y validación de paridad Python → Dart

La paridad se validó en tres niveles. Cada uno tiene su golden set JSON generado por Python y su suite de tests en Dart:

Golden set	Generado por	Qué valida	Tests
<code>models/tokenizer_golden_set.json</code>	<code>generar_golden_set.py</code>	Tokenizador: mismos tokens e IDs	37/37
<code>models/postproc_golden_set.json</code>	<code>generar_golden_set_postproc.py</code>	R1+R2 etiquetas + R3+extractor campos	53/53

Criterio de avance: no se portó ninguna lógica a Dart hasta tener 100% de paridad en Python primero, y no se exportó el TFLite hasta tener 100% de paridad tokenizador + post-procesamiento en Dart.

Archivos de modelo en `models/`

Archivo	Descripción	¿En producción?
<code>ner_clinico_es/</code>	SpaCy NER (11 entidades)	No — sin soporte TFLite
<code>isolation_forest.joblib</code>	Detector de anomalías sklearn	Sí — módulo de alertas
<code>scaler_if.joblib</code>	StandardScaler para Isolation Forest	Sí
<code>ner_tflite.keras</code>	BiLSTM NER v1	No — superado por v2
<code>ner_tflite_v2.keras</code>	BiLSTM NER v2	Sí — entrenamiento/evaluación Python
<code>ner_tflite_v2.tflite</code>	Exportación TFLite (757 KB)	Sí — on-device Flutter
<code>tflite_vocab_v2.json</code>	Vocabulario token→id (2 196 tokens)	Sí
<code>tflite_labels_v2.json</code>	Mapa label→id (14 etiquetas BIO)	Sí

Archivo	Descripción	¿En producción?
<code>tokenizer_golden_set.json</code>	Referencia de paridad tokenizador	Tests
<code>postproc_golden_set.json</code>	Referencia de paridad post-procesamiento	Tests

Entorno de desarrollo

```
Python (TF 2.15): .\env\Scripts\python.exe
Flutter:          C:\Users\Rocko43\Downloads\flutter\bin\flutter.bat
Dart:             C:\Users\Rocko43\Downloads\flutter\bin\cache\dart-
                  sdk\bin\dart.exe
```

Ejecutar los 90 tests de paridad:

```
cd flutter_bridge
C:\Users\Rocko43\Downloads\flutter\bin\flutter.bat test
```

Regenerar todo desde cero (si se reentrena el modelo):

```
.\env\Scripts\python.exe generar_dataset_tflite_v2.py
.\env\Scripts\python.exe entrenar_tflite_ner_v2.py
.\env\Scripts\python.exe generar_golden_set.py
.\env\Scripts\python.exe generar_golden_set_postproc.py
.\env\Scripts\python.exe exportar_tflite.py
```